



V1.6.11

SocketCmd通讯功能使用说明书

A0

文档版本说明

版本	A0
发布时间	2023/11/07
修订说明	修订文档并第一次发布_XTB
文档名称	《V1. 6. 11_SocketCmd 通讯功能使用说明书_A0》
软件版本	HSC3_V1.6.11 及以上版本

目录

- 1 功能简介 1
- 2 协议说明 2
 - 2.1 协议报文格式 2
 - 2.2 协议报文详解 3
 - 2.3 协议版本 3
- 3 服务端向客户端推送消息 4
 - 3.1 报警消息推送 4
- 4 终端命令列表 5
 - 4.1 程序运行代理 VM 6
 - 4.1.1 设置执行模式 6
 - 4.1.2 加载程序 6
 - 4.1.3 启动执行程序 6
 - 4.1.4 单步执行程序 6
 - 4.1.5 程序内跳转 7
 - 4.1.6 回退执行程序 7
 - 4.1.7 暂停执行程序 7
 - 4.1.8 停止执行程序 7

4.1.9	卸载程序	8
4.1.10	进入编辑程序	8
4.1.11	退出编辑程序	8
4.1.12	插入或修改一行程序内容	8
4.1.13	删除程序内容	9
4.1.14	新增或修改点位	9
4.1.15	删除点位	10
4.1.16	获取程序状态	10
4.1.17	是否已加载	11
4.1.18	获取当前行	11
4.1.19	获取当前运行程序名	11
4.1.20	获取已加载的主程序列表	11
4.1.21	获取程序信息	12
4.1.22	获取 INT 类型变量	12
4.1.23	获取 DOUBLE 类型变量	12
4.1.24	设置 DOUBLE 类型变量	13
4.2	寄存器操作代理 VAR	13
4.2.1	获取变量值	13
4.2.2	设置变量值	13
4.2.3	将变量值设置给另一个变量	14
4.2.4	保存轴数据	14
4.2.5	保存所有轴数据	14
4.2.6	保存组数据	14
4.2.7	保存所有组数据	15
4.2.8	保存公共数据	15
4.2.9	设置 R 寄存器	15
4.2.10	获取 R 寄存器	16
4.2.11	获取多个 R 寄存器	16

4.2.12	设置 JR 寄存器	16
4.2.13	获取 JR 寄存器	17
4.2.14	获取多个 JR 寄存器	17
4.2.15	设置 LR 寄存器	17
4.2.16	获取 LR 寄存器	18
4.2.17	获取多个 LR 寄存器	18
4.2.18	配置区域原点	18
4.2.19	配置区域形状	19
4.2.20	配置区域伸缩值	19
4.2.21	配置区域类型	19
4.2.22	获取区域类型	20
4.2.23	获取区域类型	20
4.2.24	配置区域输出信号反转	20
4.2.25	配置共享区处理模式	21
4.2.26	使能共享区	21
4.2.27	获取共享区使能	21
4.2.28	获取共享区使能	22
4.2.29	获取区域输出	22
4.2.30	获取区域输出	22
4.2.31	获取区域配置信息	22
4.3	传送带跟踪代理 TRACK	23
4.3.1	标定传送带	23
4.3.2	获取当前硬件触发编号	23
4.3.3	构造软件触发项	24
4.3.4	更新软件触发项	24
4.3.5	添加目标	24
4.3.6	添加带有基于 TP 点偏移参数的目标	25
4.3.7	获取位置	25

4.3.8 获取当前编码器值	26
4.4 系统功能代理 SYS.....	26
4.4.1 系统复位	26
4.4.2 系统是否存在报警	26
4.4.3 查询报警详情	27
4.4.4 系统是否就绪	27
4.4.5 准备升级	27
4.4.6 准备更新用户 PLC	27
4.4.7 注册	28
4.4.8 获取授权用 SN	28
4.4.9 授权	28
4.4.10 剩余授权时间	28
4.4.11 是否授权	29
4.4.12 设置语言	29
4.4.13 备份	29
4.4.14 恢复备份	29
4.5 伺服代理 SERVO.....	30
4.5.1 获取指定轴对应伺服的伺服参数名字列表	30
4.5.2 获取指定轴对应伺服、指定伺服参数的格式化信息	30
4.5.3 打印伺服参数信息	31
4.5.4 读指定轴对应伺服的各参数	31
4.5.5 读指定轴对应伺服的指定伺服参数	31
4.5.6 写指定轴对应伺服的指定伺服参数	31
4.6 恢复代理 RECOVER	32
4.6.1 设置恢复程序名并保存	32
4.6.2 清空恢复程序名并保存	32
4.6.3 获取恢复程序名	32
4.7 PDO 代理 PDO	33

4.7.1	设置 PDO	33
4.7.2	获取 PDO	33
4.8	网络代理 NET	33
4.8.1	获取网络连接 ID 列表	33
4.8.2	结束网络连接	34
4.8.3	获取连接信息	34
4.8.4	启用自定义网络	34
4.8.5	禁用自定义网络	35
4.8.6	获取自定义网络配置信息	35
4.8.7	保存自定义网络配置	35
4.9	运动功能代理 MOT	36
4.9.1	获取运动层版本信息	36
4.9.2	打印控制	36
4.9.3	获取打印控制设置	36
4.9.4	设置操作模式	36
4.9.5	获取操作模式	37
4.9.6	设置手动运动类型	37
4.9.7	获取手动运动类型	37
4.9.8	设置手动增量距离	37
4.9.9	获取手动增量距离	38
4.9.10	获取手动运行状态	38
4.9.11	设置自动运动倍率	38
4.9.12	获取自动运动倍率	38
4.9.13	设置手动运动倍率	39
4.9.14	获取手动运动倍率	39
4.9.15	急停	39
4.9.16	获取急停状态	39
4.9.17	组使能	40

4.9.18	获取组使能状态	40
4.9.19	组拖动示教	40
4.9.20	获取组拖动示教状态	41
4.9.21	组复位	41
4.9.22	设置使用虚轴	41
4.9.23	零点校准	41
4.9.24	获取零点	42
4.9.25	设置软限位	42
4.9.26	获取软限位	42
4.9.27	设置电流限位	43
4.9.28	获取电流限位	43
4.9.29	单轴运动开始	43
4.9.30	单轴运动停止	44
4.9.31	设置工作坐标系	44
4.9.32	获取工作坐标系	44
4.9.33	设置机型	45
4.9.34	设置负载	45
4.9.35	获取机器人类型	45
4.9.36	获取机器人类型名字	46
4.9.37	获取机器人负载	46
4.9.38	获取可用机型名字	46
4.9.39	获取内部轴数	46
4.9.40	设置附加轴数	47
4.9.41	获取附加轴数	47
4.9.42	获取附加轴起始轴号	47
4.9.43	配置协同组	47
4.9.44	获取协同组配置	48
4.9.45	获取轴组当前可协同运动轴的编号掩码	48

4.9.46	获取关节坐标点数据	49
4.9.47	获取附加轴坐标点数据	49
4.9.48	获取关节反馈电流数据	49
4.9.49	获取附加轴反馈电流数据	49
4.9.50	获取笛卡尔坐标点数据	50
4.9.51	获取形态位	50
4.9.52	检查点位是否可达	50
4.9.53	运动定位	51
4.9.54	圆弧运动定位	51
4.9.55	设置工件坐标系	52
4.9.56	获取工件坐标系	52
4.9.57	设置工具坐标系	52
4.9.58	获取工具坐标系	53
4.9.59	设置安装工具方式	53
4.9.60	获取安装工具方式	53
4.9.61	设置协同组号	54
4.9.62	获取协同组号	54
4.9.63	设置工件号	54
4.9.64	获取工件号	54
4.9.65	设置工具号	55
4.9.66	获取工具号	55
4.9.67	标定	55
4.9.68	标定附加轴减速比	56
4.9.69	获取轴减速比	56
4.9.70	20 点标定	56
4.10	业务功能代理 LIB	57
4.10.1	获取业务层版本信息	57
4.10.2	设置控制器名字	57

4.10.3	获取控制器版本信息	57
4.10.4	关机	58
4.10.5	重启	58
4.10.6	获取控制器信息	58
4.10.7	将内存数据同步至硬盘	58
4.10.8	清理硬盘数据	59
4.11	数字 IO 操作代理 IO	59
4.11.1	获取数字输入端口数量	59
4.11.2	获取数字输出端口数量	59
4.11.3	获取数字输入分组数量	60
4.11.4	获取数字输出分组数量	60
4.11.5	获取数字输入分组值	60
4.11.6	获取数字输出分组值	60
4.11.7	获取数字输入分组状态	61
4.11.8	获取数字输出分组状态	61
4.11.9	设置数字输入端口值	61
4.11.10	设置数字输出端口值	62
4.11.11	获取数字输入端口值	62
4.11.12	获取数字输出端口值	62
4.11.13	设置数字输入状态值	62
4.11.14	设置数字输出状态值	63
4.12	零点操作代理 HOME	63
4.12.1	设置零点	63
4.12.2	获取零点	63
4.12.3	通过零点偏差计算零点	64
4.12.4	获取零点偏移	64
4.12.5	通过编码器值计算零点	64
4.13	外部运行功能代理 EXT	65

4.13.1	添加外部输入信号	65
4.13.2	删除外部输入信号	65
4.13.3	获取外部输入信号配置列表	65
4.13.4	添加外部输出信号	66
4.13.5	删除外部输出信号	66
4.13.6	获取外部输出信号配置列表	66
4.13.7	修改引用寄存器配置	66
4.13.8	删除引用寄存器配置	67
4.13.9	获取引用寄存器配置	67
4.13.10	获取引用寄存器配置列表	67
4.13.11	设置外部运行程序	68
4.13.12	获取外部运行程序映射表	68
4.13.13	设置外部运行程序编号绑定寄存器	68
4.13.14	获取外部运行程序编号绑定寄存器	69
4.13.15	保存钩子相关内容	69
4.13.16	保存	69
4.14	动力学功能代理 DYN	69
4.14.1	设置协作模式	69
4.14.2	获取协作模式	70
4.14.3	初始化动力学辨识模块	70
4.14.4	设置负载辨识关节运动范围	70
4.14.5	获取负载激励轨迹系数	70
4.14.6	运行激励轨迹	71
4.14.7	低速运行激励轨迹	71
4.14.8	停止激励轨迹运行	71
4.14.9	获取辨识轨迹起始点	72
4.14.10	执行动力学辨识	72
4.14.11	执行负载辨识	72

4.14.12	获取激励轨迹运行状态	72
4.14.13	负载切换	73
4.14.14	清除负载	73
4.14.15	切换动力学模块	73
4.14.16	设置碰撞灵敏度	73
4.14.17	获取碰撞灵敏度	74
4.15	采集操作代理 COLLECT	74
4.15.1	配置采集任务	74
4.15.2	添加采集任务	74
4.15.3	删除采集任务	75
4.15.4	启动所有采集任务	75
4.15.5	启动采集任务	75
4.15.6	停止所有采集任务	75
4.15.7	停止采集任务	76
4.15.8	获取存在的采集任务列表	76
4.15.9	获取正在运行的采集任务列表	76
4.15.10	获取当前时间戳	77
4.16	编码解码功能代理 CODER	77
4.16.1	添加编码器	77
4.16.2	删除编码器	77
4.16.3	获取编码器列表	78
4.16.4	添加解码器	78
4.16.5	删除解码器	78
4.16.6	获取解码器列表	79
4.16.7	保存	79
4.17	标定操作代理 CALI	79
4.17.1	协同组标定	79
4.17.2	外部轴减速比标定	80

4.17.3	外部工具标定	80
4.17.4	外部工件标定	80
4.18	算法功能代理 ALGO	81
4.18.1	关节坐标转换空间坐标 1	81
4.18.2	关节坐标转换空间坐标 2	81
4.18.3	关节坐标转换空间坐标 3	82
4.19	模拟 IO 操作代理 AIN/AOUT	82
4.19.1	获取 IO 变量数量	82
4.19.2	获取真实端口数量	83
4.19.3	获取端口值	83
4.19.4	设置端口值	83
4.19.5	获取端口状态	83
4.19.6	设置端口状态	84
4.19.7	获取 IO 分组状态	84
4.19.8	设置 IO 分组状态	84

1 功能简介

SocketCmd 网络终端命令是机械臂控制器约定的一套字符串控制协议。上位机与机械臂建立 Socket 通讯后，发送指定的字符串即可控制机械臂完成相关的动作，或者查询机械臂系统的状态信息。

使用 SocketCmd 网络终端命令时，机械臂作为服务端，通讯端口默认是 23333。IP 地址就是机械臂控制器的 IP 地址。

IPC 电柜的控制器的 LAN0 网口的 IP 地址如下：

- 10.10.56.214

驱控一体电柜有多个网口，每个网口的 IP 地址如下：

- MLAN: 10.10.57.213
- LAN2: 10.10.58.212
- LAN1: 10.10.59.211

使用 SocketCmd 终端命令时，须把上位机和机械臂控制器用网线连接（注意所选的网口），然后在示教器/工作站软件界面上开启 SocketCmd 功能。上位机作为客户端连接机械臂控制器的 IP 和端口号，发送协议字符串即可控制机械臂。

2 协议说明

2.1 协议报文格式

上位机发送的数据报文格式如下：

i:流水号	c:终端命令	分隔符
-------	--------	-----

机械臂控制器返回的数据报文格式如下：

i:流水号	e:错误码	d:命令返回数据内容	分隔符
-------	-------	------------	-----

数据报文说明：

- (1) 数据报文的编码使用 UTF-8。
- (2) 流水号的值类型为 `uint`。值范围为 0 至 2147483648。进行中的请求中一般不能有重复的流水号，如果单个连接的通信请求是串行进行的，该流水号允许设置重复的值，例如一直设置为 0 或者其他任意数字。发送报文的流水号与返回报文的流水号相同。
- (3) 终端命令请参阅第 4 章节终端命令列表。
- (4) 错误码是一个 64 位的整型数据，表示的是请求响应的结果状态。
 - 当错误码为 0 时，表示请求响应成功。
 - 当错误码为非 0 时，表示请求响应失败。
- (5) 分隔符为 “@hs@”。此为固定格式。

数据报文示例：

终端命令功能描述	上位机请求的数据报文	机械臂响应的数据报文
机械臂上使能	i:10000,c:mot.setGpEn(0,true)@hs@	i:10000,e:0,d:null@hs@
获取机械臂使能状态	i:10001,c:mot.getGpEn(0)@hs@	i:10001,e:0,d:true@hs@
设置机械臂负载等级为重载	i:10002,c:mot.setRobLoad(0,"H")@hs@	i:10002,e:0,d:null@hs@

2.2 协议报文详解

以上述示例中的机械臂上使能功能为例：

上位机（客户端）发送：i:10000,c:mot.setGpEn(0,true)@hs@

机械臂（服务端）返回：i:10000,e:0,d:null@hs@

(1) 发送报文详解：

- i:10000。流水号。可以根据实际情况填写，不一定是 10000。可以每发送一条指令就让流水号加 1，也可以不加。
- c:mot.setGpEn(0,true)。终端命令。在此条命令中，mot 表示为运动模块（其他如 io 模块表示为“io”，寄存器操作表示为“var”等），setGpEn()为该模块下的组使能设置接口，0 和 true 为该接口的参数。其中，0 为组号，当前版本的机械臂组号都是 0，所以默认为 0 即可；true 为上使能，反之 false 为下使能。其余命令的具体接口内容可查看第 4 章节终端命令列表。
- @hs@。分隔符。为固定格式，无需更改。

(2) 返回报文详解：

- i:10000。流水号。与发送报文的流水号相同。
- e:0。错误码。此处 0 表示请求响应成功。
- d:null。命令返回数据内容。null 表示该接口为 void 返回值，也有返回为空字符的，都以 void 处理即可。
- @hs@。分隔符。为固定格式，无需更改。

2.3 协议版本

通过发送指定指令设置要使用的协议版本进行协议的切换，不设置协议版本默认使用最新协议。

设置协议版本的终端命令为：sc.setProtoVer(版本号)。

数据报文示例：i:10000,c:sc.setProtoVer(1.0)@hs@

3 服务端向客户端推送消息

服务端向客户端推送消息统一采用 2.1 章节中描述的机械臂响应的数据报文格式。

3.1 报警消息推送

客户端（上位机）可以持续监听服务端（机械臂）的 23333 端口接收来自机械臂推送的报警消息，服务端会默认向所有的可用连接通道推送报警消息，如果需要接收报警消息，建议客户端与服务端仅且保持一条可用连接通道。协议约定当服务端向客户端推送的消息中的流水号为-1 时，该消息为报警消息，报警消息的格式如下：

i:-1	e:错误码	d:报警消息内容	分隔符
------	-------	----------	-----

为了方便客户端解析，报警消息内容使用标准的 JSON 格式。

报警消息的 JSON 数据示例说明如下：

```
{
  "time":"2023-01-01 13:45:51.032"           //时间，格式 yyyy-MM-dd HH:mm:ss.SSS
  ,"errorCode":"302000c0230000"             //错误码，16 进制表示的数值的字符串
  ,"content":"[警告][运动层][0][0] 协作模式切换失败 "    //报警内容
  ,"errorLevel":3                             //报警等级
}
```

其中，errorLevel 报警等级各数值说明如下：

- 0: 代表等级 UNKNOWN
- 1: 代表等级 INFO
- 2: 代表等级 NOTE
- 3: 代表等级 WARNING
- 4: 代表等级 ERROR

4 终端命令列表

在机械臂控制系统中，终端命令各模块的说明如下：

模块名	模块功能
vm	程序运行代理。提供 PRG 程序相关功能
var	寄存器操作代理。提供 R、JR、LR 等寄存器操作、区域功能
track	传送带跟踪功能代理。提供传送带跟踪的相关操作功能
sys	系统功能代理。提供系统报警、注册授权、恢复备份等操作功能
servo	伺服操作代理。提供伺服相关的操作功能
recover	恢复操作代理。提供程序名获取、设置、清空操作功能
pdo	PDO 操作代理。提供 PDO 获取及设置操作功能
net	网络功能代理。提供网络连接、自定义网络等操作功能
mot	运动功能代理。提供轴组配置、运行操作等操作功能
lib	业务功能代理。提供版本信息、关机重启等操作功能
io	数字 IO 操作代理。提供数字 IO 端口获取、设置等操作功能
home	零点操作代理。提供零点获取、设置、计算等操作功能
ext	外部运行功能代理。提供外部运行信号、程序等操作功能
dyn	动力学功能代理。提供负载辨识、碰撞检测等操作功能
collect	采集操作代理。提供采集功能，可建立采集任务对指定的若干采集变量以指定周期连续地采集数据
coder	编码解码功能代理。提供编码器与解码器操作功能
cali	标定操作代理。提供外部工具工件、协同组、外部轴等标定操作功能
Algo	算法功能代理。提供关节坐标转换空间坐标操作功能
ain/aout	模拟 IO 操作代理。提供模拟 IO 端口获取、设置等操作功能

4.1 程序运行代理 vm

4.1.1 设置执行模式

命令格式	vm.setMode(int mode)
命令功能	设置执行模式
接口参数	mode: 模式。0 表示 NORMAL 普通模式；1 表示 OPT 优化模式
返回值	null

4.1.2 加载程序

命令格式	vm.load(string path, string entry)
命令功能	加载程序
接口参数	- path: 路径。此为保留参数，建议为空 - entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	null

4.1.3 启动执行程序

命令格式	vm.start(string entry)
命令功能	启动执行程序
接口参数	entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	null

4.1.4 单步执行程序

命令格式	vm.step(string entry)
命令功能	单步执行程序
接口参数	entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”

返回值	null
-----	------

4.1.5 程序内跳转

命令格式	vm.jump(string entry, int lineNum)
命令功能	程序内跳转
接口参数	- entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG” - lineNum: 程序行号
返回值	null

4.1.6 回退执行程序

命令格式	vm.back(string entry)
命令功能	回退执行程序
接口参数	entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	null

4.1.7 暂停执行程序

命令格式	vm.pause(string entry)
命令功能	暂停执行程序
接口参数	entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	null

4.1.8 停止执行程序

命令格式	vm.stop(string entry)
命令功能	停止执行程序

接口参数	entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	null

4.1.9 卸载程序

命令格式	vm.unload(string entry)
命令功能	卸载程序
接口参数	entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	null

4.1.10 进入编辑程序

命令格式	vm.enterEdit(string entry)
命令功能	进入编辑程序。该命令仅在 ST_READY、ST_PAUSED 状态下有效
接口参数	entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	null

4.1.11 退出编辑程序

命令格式	vm.exitEdit(string entry)
命令功能	退出编辑程序。该命令仅在 ST_WAITING 状态下有效
接口参数	entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	null

4.1.12 插入或修改一行程序内容

命令格式	vm.modifyProgLine(string entry, string progName, int lineNum, bool isInsert, string content)
命令功能	插入或修改一行程序内容。该命令仅在 ST_WAITING 状态下有效

接口参数	● entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG” ● progName: 程序文件名 ● lineNum: 程序行号 ● isInsert: 插入或修改程序。true 表示在行号处之下插入内容；false 表示将行号处内容进行替换 ● content: 一行内容
返回值	❖ 0: 表示失败 ❖ 1: 表示成功

4.1.13 删除程序内容

命令格式	vm.deleteProgLine(string entry, string progName, int lineNum, int count)
命令功能	删除程序内容。该命令仅在 ST_WAITING 状态下有效
接口参数	● entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG” ● progName: 程序文件名 ● lineNum: 程序行号 ● count: 程序行数
返回值	null

4.1.14 新增或修改点位

命令格式	vm.modifyP(string entry, string progName, int indexP, string value)
命令功能	新增或修改点位。该命令仅在 ST_WAITING 状态下有效
接口参数	● entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG” ● progName: 程序文件名 ● indexP: P 点索引。若索引对应 P 点不存在，则新增 P 点；若已存在，则修改 P 点 ● value: 点位。关节点位为“{{1.0,2.0,3.0,4.0,5.0,6.0}}”；空间点位为

	“{1,2,0,{1.0,2.0,3.0,4.0,5.0,6.0}}”，前三位表示工具号为 1，工件号为 2，形态位为 0
返回值	❖ 0：表示失败 ❖ 1：表示成功

4.1.15 删除点位

命令格式	vm.deleteP(string entry, string progName, int indexP)
命令功能	删除点位。该命令仅在 ST_WAITING 状态下有效
接口参数	● entry：入口。例如“TEST.PRG”、“/X/TEST.PRG” ● progName：程序文件名 ● indexP：P 点索引。若索引对应 P 点不存在，则新增 P 点；若已存在，则修改 P 点
返回值	null

4.1.16 获取程序状态

命令格式	vm.progState(string entry)
命令功能	获取程序状态
接口参数	● entry：入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	❖ 1：表示 ST_UNLOAD（未加载） ❖ 2：表示 ST_READY（就绪） ❖ 3：表示 ST_RUNNING（运行） ❖ 4：表示 ST_PAUSED（暂停） ❖ 7：表示 ST_FAULT（错误） ❖ 8：表示 ST_LOADING（加载中） ❖ 9：表示 ST_WAITING（等待）

4.1.17 是否已加载

命令格式	vm.isLoaded(string entry)
命令功能	是否已加载
接口参数	● entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	❖ 0: 表示未加载 ❖ 1: 表示已加载

4.1.18 获取当前行

命令格式	vm.getCurLineNo(string entry)
命令功能	获取当前行
接口参数	● entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	❖ 1 至 n: 表示行号

4.1.19 获取当前运行程序名

命令格式	vm.getCurProgName(string entry)
命令功能	获取当前运行程序名
接口参数	● entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	程序名

4.1.20 获取已加载的主程序列表

命令格式	vm.mainProgNames()
命令功能	获取已加载的主程序列表
接口参数	null
返回值	程序列表。格式：中括号扩起，半角逗号分隔的字符串队列 例如 “[ABC.PRG, DEF.PRG]”

4.1.21 获取程序信息

命令格式	vm.getProgInfo(string entry)
命令功能	获取程序信息
接口参数	● entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG”
返回值	信息内容 例如“ret=0, cur_prog="****", line_no=0, back_prog="****", line_of_back=0, state=1, uf_num=-1, ut_num=-1” 其中：“ret”是获取返回值，“cur_prog”是当前程序名，“line_no”是行号，“back_prog”是后退指针指向程序名，“line_of_back”是后退指针指向行号，“state”是运行状态，“uf_num”是运行时工件号，“ut_num”是运行时工具号

4.1.22 获取 int 类型变量

命令格式	vm.getVariableOfInt(string entry, string npName, string varName)
命令功能	获取 int 类型变量
接口参数	● entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG” ● npName: 变量空间名。此为保留参数，可为空 ● varName: 变量名。例如“UFRAME_NUM”、“CNT”、“J_VEL”等
返回值	变量值

4.1.23 获取 double 类型变量

命令格式	vm.getVariableOfDouble(string entry, string npName, string varName)
命令功能	获取 double 类型变量
接口参数	● entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG” ● npName: 变量空间名。此为保留参数，可为空 ● varName: 变量名。例如“UFRAME_NUM”、“CNT”、“J_VEL”等

返回值	变量值
-----	-----

4.1.24 设置 double 类型变量

命令格式	vm.setVariableOfDouble(string entry, string npName, string varName, double value)
命令功能	设置 double 类型变量
接口参数	● entry: 入口。例如“TEST.PRG”、“/X/TEST.PRG” ● npName: 变量空间名。此为保留参数，可为空 ● varName: 变量名。例如“UFRAME_NUM”、“CNT”、“J_VEL”等 ● value: 变量值
返回值	null

4.2 寄存器操作代理 var

4.2.1 获取变量值

命令格式	var.getVarValue(string strVarFullName)
命令功能	获取变量值
接口参数	● strVarFullName: 变量完整名。例如“group[0].tfnum”
返回值	值

4.2.2 设置变量值

命令格式	var.setVarValue(string strVarFullName, string strValue)
命令功能	设置变量值
接口参数	● strVarFullName: 变量完整名。例如“group[0].tfnum” ● strValue: 值
返回值	null

4.2.3 将变量值设置给另一个变量

命令格式	<code>var.setVarValueByVar(string strVarFullName, string strSrcVarFullName)</code>
命令功能	将变量值设置给另一个变量
接口参数	● <code>strVarFullName</code>: 变量完整名。例如 “group[0].tfnum” ● <code>strSrcVarFullName</code>: 另一个变量完整名。例如 “group[1].tfnum”
返回值	null

4.2.4 保存轴数据

命令格式	<code>var.saveAx(int8_t axId, string file)</code>
命令功能	保存轴数据
接口参数	● <code>axId</code>: 轴号。范围为 0 至 n-1 ● <code>file</code>: 文件名。<code>para</code> 表示参数；<code>home</code> 表示原点相关参数；<code>limit</code> 表示软限位相关参数
返回值	null

4.2.5 保存所有轴数据

命令格式	<code>var.saveAx(string file)</code>
命令功能	保存所有轴数据
接口参数	● <code>file</code>: 文件名。<code>para</code> 表示参数；<code>home</code> 表示原点相关参数；<code>limit</code> 表示软限位相关参数
返回值	null

4.2.6 保存组数据

命令格式	<code>var.saveGp(int8_t gpId, string file)</code>
命令功能	保存组数据

接口参数	● gpId: 组号。范围为 0 至 n-1 ● file: 文件名。para 表示参数；JR 表示 JR 寄存器值；LR 表示 LR 寄存器值；UT 表示 UT 寄存器值；UF 表示 UF 寄存器值；area 表示区域相关参数；track 表示跟踪相关参数
返回值	null

4.2.7 保存所有组数据

命令格式	var.saveGp(string file)
命令功能	保存所有组数据
接口参数	● file: 文件名。para 表示参数；JR 表示 JR 寄存器值；LR 表示 LR 寄存器值；UT 表示 UT 寄存器值；UF 表示 UF 寄存器值；area 表示区域相关参数；track 表示跟踪相关参数
返回值	null

4.2.8 保存公共数据

命令格式	var.saveCommon(string file)
命令功能	保存公共数据
接口参数	● file: 文件名。R 表示 R 寄存器值；system 表示系统参数；mb 表示 Modbus 参数
返回值	null

4.2.9 设置 R 寄存器

命令格式	var.setR(int32_t index, double value)
命令功能	设置 R 寄存器
接口参数	● index: 索引。范围为 0 至 n-1

	● value: 浮点数值
返回值	null

4.2.10 获取 R 寄存器

命令格式	var.getR(int32_t index)
命令功能	获取 R 寄存器
接口参数	● index: 索引。范围为 0 至 n-1
返回值	浮点数值

4.2.11 获取多个 R 寄存器

命令格式	var.getR(int32_t index, int32_t len)
命令功能	获取多个 R 寄存器
接口参数	● index: 起始索引。范围为 0 至 n-1 ● len: 数量
返回值	浮点数值列表。格式：中括号扩起、半角逗号分隔的字符串队列 例如：“[1.0, 2.0]”

4.2.12 设置 JR 寄存器

命令格式	var.setJR(int8_t gpId, int32_t index, string value)
命令功能	设置 JR 寄存器
接口参数	● gpId: 组号。范围为 0 至 n-1 ● index: 索引。范围为 0 至 n-1 ● value: 点数据。格式：大括号扩起、半角逗号分隔的数列。例如：“{{7,8}}”
返回值	null

4.2.13 获取 JR 寄存器

命令格式	var.getLR(int8_t gpId, int32_t index)
命令功能	获取 JR 寄存器
接口参数	● gpId: 组号。范围为 0 至 n-1 ● index: 索引。范围为 0 至 n-1
返回值	点数据。格式：大括号扩起、半角逗号分隔的数列 例如：“{{7,8}}”

4.2.14 获取多个 JR 寄存器

命令格式	var.getJR(int8_t gpId, int32_t index, int32_t len)
命令功能	获取多个 JR 寄存器
接口参数	● gpId: 组号。范围为 0 至 n-1 ● index: 起始索引。范围为 0 至 n-1 ● len: 数量
返回值	点数据列表。格式：中括号扩起、半角逗号分隔的关节坐标数据队列 例如：“{{7,8}}”

4.2.15 设置 LR 寄存器

命令格式	var.setLR(int8_t gpId, int32_t index, string value)
命令功能	设置 LR 寄存器
接口参数	● gpId: 组号。范围为 0 至 n-1 ● index: 索引。范围为 0 至 n-1 ● value: 点数据。例如：“{1,2,3,{7,8}}”，其中“1,2,3,”分别是工具号、工件号、形态位，“{7,8}”是点数据，是大括号扩起、半角逗号分隔的数列
返回值	null

4.2.16 获取 LR 寄存器

命令格式	var.getLR(int8_t gpId, int32_t index)
命令功能	获取 LR 寄存器
接口参数	● gpId: 组号。范围为 0 至 n-1 ● index: 索引。范围为 0 至 n-1
返回值	点数据 例如：“{1,2,3,{7,8}}”，其中“1,2,3,”分别是工具号、工件号、形态位，“{7,8}”是点数据，是大括号扩起、半角逗号分隔的数列

4.2.17 获取多个 LR 寄存器

命令格式	var.getLR(int8_t gpId, int32_t index, int32_t len)
命令功能	获取多个 LR 寄存器
接口参数	● gpId: 组号。范围为 0 至 n-1 ● index: 索引。范围为 0 至 n-1 ● len: 数量
返回值	点数据列表。格式：中括号扩起、半角逗号分隔的笛卡尔坐标数据队列 例如：“[{1,2,3,{7,8}}], [{1,2,3,{7,8}}]”

4.2.18 配置区域原点

命令格式	var.setAreaOrigin(int8_t gpId, int8_t iIndex, int8_t ufNum, string strOrigin)
命令功能	配置区域原点
接口参数	● gpId: 组号。范围为 0 至 n-1 ● iIndex: 区域索引。范围为 0 至 n-1 ● ufNum: 原点相对工件号。范围为-1 至 15 ● strOrigin: 原点数据 XYZ。格式：大括号扩起、半角逗号分隔的数列。例如 “{1.0,2.0,3.0}”

返回值	null
-----	------

4.2.19 配置区域形状

命令格式	var.setAreaShape(int8_t gpId, int8_t iIndex, int8_t type, string strSize)
命令功能	配置区域形状
接口参数	● gpId: 组号。范围为 0 至 n-1 ● iIndex: 区域索引。范围为 0 至 n-1 ● type: 形状类型 ● strSize: 形状数据。格式：大括号扩起、半角逗号分隔的数列。例如 “{1.0,2.0,3.0,4.0}”
返回值	null

4.2.20 配置区域伸缩值

命令格式	var.setAreaOffset(int8_t gpId, int8_t iIndex, float offset)
命令功能	配置区域伸缩值
接口参数	● gpId: 组号。范围为 0 至 n-1 ● iIndex: 区域索引。范围为 0 至 n-1 ● offset: 伸缩值。单位：mm
返回值	null

4.2.21 配置区域类型

命令格式	var.setAreaType(int8_t gpId, int8_t iIndex, int8_t type)
命令功能	配置区域类型
接口参数	● gpId: 组号。范围为 0 至 n-1 ● iIndex: 区域索引。范围为 0 至 n-1

	● type: 区域类型
返回值	null

4.2.22 获取区域类型

命令格式	var.getAreaType(int8_t gpId, int8_t iIndex)
命令功能	获取区域类型
接口参数	● gpId: 组号。范围为 0 至 n-1 ● iIndex: 区域索引。范围为 0 至 n-1
返回值	区域类型

4.2.23 获取区域类型

命令格式	var.getAreaType(int8_t gpId)
命令功能	获取区域类型
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	数据列表。格式：半角逗号分隔的数据队列 例如：“{1, 2, ...}”

4.2.24 配置区域输出信号反转

命令格式	var.setAreaOutRev(int8_t gpId, int8_t iIndex, bool en)
命令功能	配置区域输出信号反转
接口参数	● gpId: 组号。范围为 0 至 n-1 ● iIndex: 区域索引。范围为 0 至 n-1 ● en: 是否反转
返回值	null

4.2.25 配置共享区处理模式

命令格式	var.setShareAreaMode(int8_t gpId, int8_t iIndex, int8_t mode)
命令功能	配置共享区处理模式
接口参数	● gpId: 组号。范围为 0 至 n-1 ● iIndex: 区域索引。范围为 0 至 n-1 ● mode: 处理模式
返回值	null

4.2.26 使能共享区

命令格式	var.setShareAreaEn(int8_t gpId, int8_t iIndex, bool en)
命令功能	使能共享区
接口参数	● gpId: 组号。范围为 0 至 n-1 ● iIndex: 区域索引。范围为 0 至 n-1 ● en: 是否使能
返回值	null

4.2.27 获取共享区使能

命令格式	var.getShareAreaEn(int8_t gpId, int8_t iIndex)
命令功能	获取共享区使能
接口参数	● gpId: 组号。范围为 0 至 n-1 ● iIndex: 区域索引。范围为 0 至 n-1
返回值	❖ 0: 表示未使能 ❖ 1: 表示已使能

4.2.28 获取共享区使能

命令格式	var.getShareAreaEn(int8_t gpId)
命令功能	获取共享区使能
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	数据列表。格式：半角逗号分隔的数据队列 例如：“{true, false, ...}”

4.2.29 获取区域输出

命令格式	var.getAreaOut(int8_t gpId, int8_t iIndex)
命令功能	获取区域输出
接口参数	● gpId: 组号。范围为 0 至 n-1 ● iIndex: 区域索引。范围为 0 至 n-1
返回值	❖ 0: 表示未输出 ❖ 1: 表示已输出

4.2.30 获取区域输出

命令格式	var.getAreaOut(int8_t gpId)
命令功能	获取区域输出
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	数据列表。格式：半角逗号分隔的数据队列 例如：“{true, false, ...}”

4.2.31 获取区域配置信息

命令格式	var.getAreaConfig(int8_t gpId, int8_t iIndex)
命令功能	获取区域配置信息

接口参数	● gpId: 组号。范围为 0 至 n-1 ● iIndex: 区域索引。范围为 0 至 n-1
返回值	信息 例如: “ufnum="0", origin="{1.0,2.0,3.0,}", shape_type="0", size="{1.0,2.0,3.0,4.0,}", offset="1.0", type="0", out_rev="false", share_mode="0"” 其中, “ufnum”是工件号, “origin”是原点数据, “shape_type”是形状类型, “size”是形状尺寸, “offset”是伸缩值, “type”是区域类型, “out_rev”是区域输出信号反转, “share_mode”是共享区处理模式

4.3 传送带跟踪代理 track

4.3.1 标定传送带

命令格式	track.calibrateConveyer(int8_t gpId, int8_t trackId, int8_t type, string strData)
命令功能	标定传送带。标定成功后数据会设置至内存, 但不会保存至文件
接口参数	● gpId: 组号。范围为 0 至 n-1 ● trackId: 跟踪号。范围为 0 至 n-1 ● type: 标定方法编号 ● strData: 标定数据。格式: 半角分号分隔的点数据队列。例如: “{1,2,};{3,4,};{5,6,}”
返回值	标定结果 例如: “ret=0x0, data={7,8,}”, 其中“ret”是标定返回值, “data”是标定结果

4.3.2 获取当前硬件触发编号

命令格式	track.getHardTrig(int8_t gpId, int8_t trackId)
命令功能	获取当前硬件触发编号。用于确保视觉与控制器处理的是同一硬件触发
接口参数	● gpId: 组号。范围为 0 至 n-1

	● trackId: 跟踪号。范围为 0 至 n-1
返回值	硬件触发编号。正常范围为 1 至 255

4.3.3 构造软件触发项

命令格式	track.createSoftTrig(int8_t gpId, int8_t trackId)
命令功能	构造软件触发项。用于确保视觉与控制器处理的是同一软件触发
接口参数	● gpId: 组号。范围为 0 至 n-1 ● trackId: 跟踪号。范围为 0 至 n-1
返回值	软件触发编号。正常范围为 1 至 255

4.3.4 更新软件触发项

命令格式	track.updateSoftTrig(int8_t gpId, int8_t trackId, int32_t trigId)
命令功能	更新软件触发项。在调用 createSoftTrig 之后调用此命令，用于减小网络通信延时对编码器值的影响
接口参数	● gpId: 组号。范围为 0 至 n-1 ● trackId: 跟踪号。范围为 0 至 n-1 ● trigId: 软件触发编号
返回值	null

4.3.5 添加目标

命令格式	track.addTarget(int8_t gpId, int8_t trackId, string data, int32_t type, int32_t trigId)
命令功能	添加目标。给定正确的触发编号，确保视觉与控制器处理的是同一硬件触发/软件触发
接口参数	● gpId: 组号。范围为 0 至 n-1 ● trackId: 跟踪号。范围为 0 至 n-1

	● data: 目标位置。格式：大括号扩起，半角逗号分隔的数列。例如： “{1.0,2.0,3.0,4.0,5.0,6.0,}”，对应 xyzabc ● type: 目标类型 ● trigId: 硬件触发/软件触发编号
返回值	null

4.3.6 添加带有基于 TP 点偏移参数的目标

命令格式	track.addTargetWithOffset(int8_t gpId, int8_t trackId, string data, string offset, int32_t type, int32_t trigId)
命令功能	添加带有基于 TP 点偏移参数的目标。给定正确的触发编号，确保视觉与控制器处理的是同一硬件触发/软件触发
接口参数	● gpId: 组号。范围为 0 至 n-1 ● trackId: 跟踪号。范围为 0 至 n-1 ● data: 目标位置。格式：大括号扩起，半角逗号分隔的数列。例如 “{1.0,2.0,3.0,4.0,5.0,6.0,}”，对应 xyzabc ● offset: 跟踪目标位置数据的偏移值。格式：大括号扩起，半角逗号分隔的数列。 例如：“{1.0,2.0,3.0,4.0,5.0,6.0,}”，对应 xyzabc 的偏移值 ● type: 目标类型 ● trigId: 硬件触发/软件触发编号
返回值	null

4.3.7 获取位置

命令格式	track.getPosition(int8_t gpId, int8_t trackId, int32_t trigId)
命令功能	获取位置。给定正确的软件触发编号，返回位置是相对于某次软件触发记录的传送带位置而言的
接口参数	● gpId: 组号。范围为 0 至 n-1

	● trackId: 跟踪号。范围为 0 至 n-1 ● trigId: 软件触发编号
返回值	字符串格式的位置数据。格式：大括号扩起，半角逗号分隔的数列 例如：“{1.0,2.0,3.0,4.0,5.0,6.0,}”，对应 xyzabc

4.3.8 获取当前编码器值

命令格式	track.getCurrentEncoder(int8_t gpId, int8_t trackId)
命令功能	获取当前编码器值
接口参数	● gpId: 组号。范围为 0 至 n-1 ● trackId: 跟踪号。范围为 0 至 n-1
返回值	编码器值

4.4 系统功能代理 sys

4.4.1 系统复位

命令格式	sys.reset()
命令功能	系统复位
接口参数	null
返回值	null

4.4.2 系统是否存在报警

命令格式	sys.hasError()
命令功能	系统是否存在报警
接口参数	null
返回值	报警数量

4.4.3 查询报警详情

命令格式	sys.queryError(uint64_t errCode)
命令功能	查询报警详情
接口参数	● errCode: 错误码
返回值	报警详情 例如: “reason="xxx", elim="yyy"”, 其中 “reason” 是原因, “elim” 是排除措施

4.4.4 系统是否就绪

命令格式	sys.isReady()
命令功能	系统是否就绪
接口参数	null
返回值	❖ 0: 表示未就绪 ❖ 1: 表示已就绪

4.4.5 准备升级

命令格式	sys.prepareUpgrade()
命令功能	准备升级。记录机型、负载等信息, 以备升级后做参数适配
接口参数	null
返回值	null

4.4.6 准备更新用户 PLC

命令格式	sys.prepareUserPlc()
命令功能	准备更新用户 PLC
接口参数	null
返回值	null

4.4.7 注册

命令格式	sys.register(string strKey, string strValue)
命令功能	注册
接口参数	● strKey: 键 ● strValue: 值
返回值	null

4.4.8 获取授权用 SN

命令格式	sys.genSn()
命令功能	获取授权用 SN
接口参数	null
返回值	sn 码

4.4.9 授权

命令格式	sys.authorize(string strCode)
命令功能	授权
接口参数	● strCode: 代码
返回值	null

4.4.10 剩余授权时间

命令格式	sys.getRemainTime()
命令功能	剩余授权时间
接口参数	null
返回值	时间。单位：秒

4.4.11 是否授权

命令格式	sys.isAuthorized()
命令功能	是否授权
接口参数	null
返回值	❖ 0: 表示未授权 ❖ 1: 表示已授权

4.4.12 设置语言

命令格式	sys.setLang(string strLang)
命令功能	设置语言
接口参数	● strLang: 语言名。"ch"表示中文; "en"表示英文
返回值	❖ 0: 表示设置失败 ❖ 1: 表示设置成功

4.4.13 备份

命令格式	sys.backupFiles(string strIntent)
命令功能	备份
接口参数	● strIntent: 意图。"MANU_CALIB_DATA"表示出厂标定参数
返回值	备份内容打包的文件名字

4.4.14 恢复备份

命令格式	sys.recoverFiles(string strIntent, string strFileName)
命令功能	恢复备份
接口参数	● strIntent: 意图。"MANU_CALIB_DATA"表示出厂标定参数

	● strFileName: 备份内容打包的文件名字
返回值	null

4.5 伺服代理 servo

4.5.1 获取指定轴对应伺服的伺服参数名字列表

命令格式	servo.getServoParaNameList(int8_t gpId, int8_t axIndex)
命令功能	获取指定轴对应伺服的伺服参数名字列表
接口参数	● gpId: 组号。范围为 0 至 n-1 ● axIndex: 组内轴索引。范围为 0 至 n-1
返回值	名字列表。格式：升序排序的半角逗号分隔的数据队列 例如：“["P005", "P006", "P007"]”

4.5.2 获取指定轴对应伺服、指定伺服参数的格式化信息

命令格式	servo.getServoParaInfoFormat(int8_t gpId, int8_t axIndex, string paraName)
命令功能	获取指定轴对应伺服、指定伺服参数的格式化信息
接口参数	● gpId: 组号。范围为 0 至 n-1 ● axIndex: 组内轴索引。范围为 0 至 n-1 ● paraName: 参数名字
返回值	信息 例如：“name="P005", chName="参数", unitName="mm", minValue=1.0, maxValue=1000.0” 其中，“name”是参数英文名，“chName”是参数中文名，“unitName”是单位名，“minValue”是最小值，“maxValue”是最大值

4.5.3 打印伺服参数信息

命令格式	<code>servo.printServoParaInfo(int8_t gpId, int8_t axIndex)</code>
命令功能	打印伺服参数信息
接口参数	● <code>gpId</code>: 组号。范围为 0 至 <code>n-1</code> ● <code>axIndex</code>: 组内轴索引。范围为 0 至 <code>n-1</code>
返回值	罗列指定轴对应伺服的各伺服参数的格式化信息

4.5.4 读指定轴对应伺服的各参数

命令格式	<code>servo.readServoPara(int8_t gpId, int8_t axIndex)</code>
命令功能	读指定轴对应伺服的各参数
接口参数	● <code>gpId</code>: 组号。范围为 0 至 <code>n-1</code> ● <code>axIndex</code>: 组内轴索引。范围为 0 至 <code>n-1</code>
返回值	值数列。格式: 半角逗号分隔的数据队列 例如: “[1.0, 1000.0, NaN]”, 对应参数按名字升序排序

4.5.5 读指定轴对应伺服的指定伺服参数

命令格式	<code>servo.readServoPara(int8_t gpId, int8_t axIndex, string paraName)</code>
命令功能	读指定轴对应伺服的指定伺服参数
接口参数	● <code>gpId</code>: 组号。范围为 0 至 <code>n-1</code> ● <code>axIndex</code>: 组内轴索引。范围为 0 至 <code>n-1</code> ● <code>paraName</code>: 参数名字
返回值	值

4.5.6 写指定轴对应伺服的指定伺服参数

命令格式	<code>servo.writeServoPara(int8_t gpId, int8_t axIndex, string paraName, double value)</code>
------	---

命令功能	写指定轴对应伺服的指定伺服参数
接口参数	● gpId: 组号。范围为 0 至 n-1 ● axIndex: 组内轴索引。范围为 0 至 n-1 ● paraName: 参数名字 ● value: 值
返回值	null

4.6 恢复代理 recover

4.6.1 设置恢复程序名并保存

命令格式	recover.setAndSaveRecoverProg(string entry)
命令功能	设置恢复程序名并保存
接口参数	● entry: 入口
返回值	null

4.6.2 清空恢复程序名并保存

命令格式	recover.clearAndSaveRecoverProg()
命令功能	清空恢复程序名并保存
接口参数	null
返回值	null

4.6.3 获取恢复程序名

命令格式	recover.getRecoverProg()
命令功能	获取恢复程序名
接口参数	null

返回值	恢复程序名
-----	-------

4.7 PDO 代理 pdo

4.7.1 设置 PDO

命令格式	pdo.setPDO(int id, int padr, int sadr, double data)
命令功能	设置 PDO
接口参数	● id: 轴号 ● padr: 一级地址 ● sadr: 二级地址 ● data: PDO 值
返回值	null

4.7.2 获取 PDO

命令格式	pdo.getPDO(int id, int padr, int sadr)
命令功能	获取 PDO
接口参数	● id: 轴号 ● padr: 一级地址 ● sadr: 二级地址
返回值	PDO 值

4.8 网络代理 net

4.8.1 获取网络连接 ID 列表

命令格式	net.getSessionIdList()
命令功能	获取网络连接 ID 列表

接口参数	null
返回值	网络连接 ID 列表。格式：中括号扩起，半角逗号分隔的整数队列 例如：“[1, 2]”

4.8.2 结束网络连接

命令格式	net.killSession(int64_t iSessionId)
命令功能	结束网络连接
接口参数	● iSessionId: 网络连接 ID
返回值	null

4.8.3 获取连接信息

命令格式	net.getSessionInfo(int64_t iSessionId)
命令功能	获取连接信息
接口参数	● iSessionId: 网络连接 ID
返回值	信息 例如：“type=“TCP”, ip=“/123.234.34.45”, port=“12345”” 其中，“type”是类型，“ip”是 IP 地址，“port”是端口号

4.8.4 启用自定义网络

命令格式	net.enableCustomedNetwork(int8_t index, string strIp, string strMask)
命令功能	启用自定义网络。重启生效
接口参数	● index: 索引号。范围为 0 和 1 ● strIp: IP 地址 ● strMask: 子网掩码
返回值	null

4.8.5 禁用自定义网络

命令格式	<code>net.disableCustomedNetwork(int8_t index)</code>
命令功能	禁用自定义网络。重启生效
接口参数	● <code>index</code>: 索引号。范围为 0 和 1
返回值	<code>null</code>

4.8.6 获取自定义网络配置信息

命令格式	<code>net.getCustomedNetworkConfigInfo()</code>
命令功能	获取自定义网络配置信息
接口参数	<code>null</code>
返回值	JSON 格式配置信息 例如: <code>{"configList":[{"en":true, "ip":"192.168.0.2", "mask":"255.255.255.0"}, ...]}</code> 其中, "<code>configList</code>"表示配置信息列表, "<code>en</code>"表示自定义网络是否启用 (<code>false</code> 表示禁用, <code>true</code> 表示启用), "<code>ip</code>"表示 IP 地址, "<code>mask</code>"表示子网掩码

4.8.7 保存自定义网络配置

命令格式	<code>net.saveCustomedNetworkConfig()</code>
命令功能	保存自定义网络配置
接口参数	<code>null</code>
返回值	<code>null</code>

4.9 运动功能代理 mot

4.9.1 获取运动层版本信息

命令格式	mot.getMotionVer()
命令功能	获取运动层版本信息
接口参数	null
返回值	版本信息

4.9.2 打印控制

命令格式	mot.setDebugLog(int32_t mode, bool flag)
命令功能	打印控制
接口参数	● mode: 模式 ● flag: 是否设置
返回值	null

4.9.3 获取打印控制设置

命令格式	mot.getDebugLog(int32_t mode)
命令功能	获取打印控制设置
接口参数	● mode: 模式
返回值	❖ 0 表示未设置 ❖ 1 表示已设置

4.9.4 设置操作模式

命令格式	mot.setOpMode(int8_t mode)
命令功能	设置操作模式

接口参数	● mode: 模式。1 表示手动 T1 模式；2 表示手动 T2 模式；3 表示自动运行模式；4 表示外部运行模式
返回值	null

4.9.5 获取操作模式

命令格式	mot.getOpMode()
命令功能	获取操作模式
接口参数	null
返回值	模式。1 表示手动 T1 模式；2 表示手动 T2 模式；3 表示自动运行模式；4 表示外部运行模式

4.9.6 设置手动运动类型

命令格式	mot.setManualMode(int8_t mode)
命令功能	设置手动运动类型
接口参数	● mode: 类型。0 表示连续运动；1 表示寸动运动
返回值	null

4.9.7 获取手动运动类型

命令格式	mot.getManualMode()
命令功能	获取手动运动类型
接口参数	null
返回值	类型。0 表示连续运动；1 表示寸动运动

4.9.8 设置手动增量距离

命令格式	mot.setInchLen(double mode)
------	-----------------------------

命令功能	设置手动增量距离
接口参数	len: 距离大小。单位根据坐标系确定，关节为°；笛卡尔为 mm
返回值	null

4.9.9 获取手动增量距离

命令格式	mot.getInchLen()
命令功能	获取手动增量距离
接口参数	null
返回值	距离大小。单位根据坐标系确定，关节为°；笛卡尔为 mm

4.9.10 获取手动运行状态

命令格式	mot.getManualStat()
命令功能	获取手动运行状态
接口参数	null
返回值	状态。0 表示停止；1 表示加速；2 表示匀速；3 表示减速

4.9.11 设置自动运动倍率

命令格式	mot.setVord(int32_t vord)
命令功能	设置自动运动倍率
接口参数	vord: 倍率。范围为 1 至 100，单位为%
返回值	null

4.9.12 获取自动运动倍率

命令格式	mot.getVord()
------	---------------

命令功能	获取自动运动倍率
接口参数	null
返回值	倍率。范围为 1 至 100，单位为%

4.9.13 设置手动运动倍率

命令格式	mot.setJogVord(int32_t vord)
命令功能	设置手动运动倍率
接口参数	● vord: 倍率。范围为 1 至 100，单位为%
返回值	null

4.9.14 获取手动运动倍率

命令格式	mot.getJogVord()
命令功能	获取手动运动倍率
接口参数	null
返回值	倍率。范围为 1 至 100，单位为%

4.9.15 急停

命令格式	mot.setEstop(bool en)
命令功能	急停
接口参数	● en: 使能。true 表示打开；false 表示关闭
返回值	null

4.9.16 获取急停状态

命令格式	mot.getEstop()
------	----------------

命令功能	获取急停状态
接口参数	null
返回值	❖ 0: 表示关闭 ❖ 1: 表示打开

4.9.17 组使能

命令格式	mot.setGpEn(int8_t gpId, bool en)
命令功能	组使能
接口参数	● gpId: 组号。范围为 0 至 n-1 ● en: 使能。true 表示打开；false 表示关闭
返回值	null

4.9.18 获取组使能状态

命令格式	mot.getGpEn(int8_t gpId)
命令功能	获取组使能状态
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	❖ 0: 表示关闭状态 ❖ 1: 表示打开状态

4.9.19 组拖动示教

命令格式	mot.setGpDrag(int8_t gpId, bool en)
命令功能	组拖动示教
接口参数	● gpId: 组号。范围为 0 至 n-1 ● en: 使能。true 表示打开；false 表示关闭
返回值	null

4.9.20 获取组拖动示教状态

命令格式	mot.getGpDrag(int8_t gpId)
命令功能	获取组拖动示教状态
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	❖ 0: 表示关闭状态 ❖ 1: 表示打开状态

4.9.21 组复位

命令格式	mot.gpReset(int8_t gpId)
命令功能	组复位
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	null

4.9.22 设置使用虚轴

命令格式	mot.setUseSimulate(int8_t gpId, bool en)
命令功能	设置使用虚轴
接口参数	● gpId: 组号。范围为 0 至 n-1 ● en: 开关
返回值	null

4.9.23 零点校准

命令格式	mot.setHomePosition(int8_t gpId, string strPos, int32_t mask)
命令功能	零点校准
接口参数	● gpId: 组号。范围为 0 至 n-1

	● strPos: 零点设定值。例如: {0, -90, 0, 0, 90, 0} ● mask: 使能掩码。低 9 位有效, 最低位代表组中第 1 个轴, 第 9 位代表组中第 9 个轴
返回值	null

4.9.24 获取零点

命令格式	mot.getHomePosition(int8_t gpId)
命令功能	获取零点
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	点数据。格式: 大括号扩起, 半角逗号分隔的数列 例如: “{1.0,2.0,3.0,4.0,5.0,6.0,}”

4.9.25 设置软限位

命令格式	mot.setJointLimit(int8_t gpId, string strLimitPos, string strLimitNeg, int32_t mask)
命令功能	设置软限位
接口参数	● gpId: 组号。范围为 0 至 n-1 ● strLimitPos: 正向设定值。例如 {90, 90, 90, 90, 90, 90} ● strLimitNeg: 负向设定值。例如 {-90, -90, -90, -90, -90, -90} ● mask: 使能掩码。低 9 位有效, 最低位代表组中第 1 个轴, 第 9 位代表组中第 9 个轴
返回值	null

4.9.26 获取软限位

命令格式	mot.getJointLimit(int8_t gpId)
命令功能	获取软限位

接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	限位信息 例如: “pos={1,1,}, neg={-1,-1,}, mask=0x1ff” 其中, “pos”是正向设定值, “neg”是负向设定值, “mask”是使能掩码

4.9.27 设置电流限位

命令格式	mot.setELimit(int8_t gpId, string strEPos, string strENeg, int32_t mask)
命令功能	设置电流限位
接口参数	● gpId: 组号。范围为 0 至 n-1 ● strLimitPos: 正向设定值。例如 {90, 90, 90, 90, 90, 90} ● strLimitNeg: 负向设定值。例如 {-90, -90, -90, -90, -90, -90} ● mask: 使能掩码。低 9 位有效, 最低位代表组中第 1 个轴, 第 9 位代表组中第 9 个轴
返回值	null

4.9.28 获取电流限位

命令格式	mot.getELimit(int8_t gpId)
命令功能	获取电流限位
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	限位信息 例如: “pos={1,1,}, neg={-1,-1,}, mask=0x1ff” 其中, “pos”是正向设定值, “neg”是负向设定值, “mask”是使能掩码

4.9.29 单轴运动开始

命令格式	mot.startJog(int8_t gpId, int8_t axId, int8_t direc)
------	--

命令功能	单轴运动开始
接口参数	● gpId: 组号。范围为 0 至 n-1 ● axId: 轴号。0 至 5 为内部轴；6 至 8 为附加轴 ● direc: 方向。0 表示正向；1 表示负向
返回值	null

4.9.30 单轴运动停止

命令格式	mot.stopJog(int8_t gpId)
命令功能	单轴运动停止
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	null

4.9.31 设置工作坐标系

命令格式	mot.setWorkFrame(int8_t gpId, int8_t frame)
命令功能	设置工作坐标系
接口参数	● gpId: 组号。范围为 0 至 n-1 ● frame: 工作坐标系。0 表示轴坐标系；1 表示世界坐标系；2 表示用户坐标系；3 表示工具坐标系
返回值	null

4.9.32 获取工作坐标系

命令格式	mot.getWorkFrame(int8_t gpId)
命令功能	获取工作坐标系
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	❖ 0: 表示轴坐标系

	❖ 1: 表示世界坐标系 ❖ 2: 表示用户坐标系 ❖ 3: 表示工具坐标系
--	--

4.9.33 设置机型

命令格式	mot.setRobType(int8_t gpId, string strTypeName, string strLoadType)
命令功能	设置机型。重启系统生效
接口参数	● gpId: 组号。范围为 0 至 n-1 ● strTypeName: 机型名字 ● strLoadType: 负载名字。"H"表示重载；"M"表示中载；"L"表示轻载
返回值	null

4.9.34 设置负载

命令格式	mot.setRobLoad(int8_t gpId, string strLoadType)
命令功能	设置负载。重启系统生效
接口参数	● gpId: 组号。范围为 0 至 n-1 ● strLoadType: 负载名字。"H"表示重载；"M"表示中载；"L"表示轻载
返回值	null

4.9.35 获取机器人类型

命令格式	mot.getRobType(int8_t gpId)
命令功能	获取机器人类型
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	机器人类型

4.9.36 获取机器人类型名字

命令格式	mot.getRobTypeName(int8_t gpId)
命令功能	获取机器人类型名字
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	机器人类型名字

4.9.37 获取机器人负载

命令格式	mot.getRobLoad(int8_t gpId)
命令功能	获取机器人负载
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	❖ "H": 重载 ❖ "M": 中载 ❖ "L": 轻载

4.9.38 获取可用机型名字

命令格式	mot.getRobTypeNameList()
命令功能	获取可用机型名字
接口参数	null
返回值	机型名字列表。格式：半角逗号分隔的数据队列 例如：“["PUMA", "SCARA"]”

4.9.39 获取内部轴数

命令格式	mot.getRobAxisCount(int8_t gpId)
命令功能	获取内部轴数
接口参数	● gpId: 组号。范围为 0 至 n-1

返回值	轴数
-----	----

4.9.40 设置附加轴数

命令格式	<code>mot.setAuxAxisCount(int8_t gpId, int32_t cnt)</code>
命令功能	设置附加轴数
接口参数	● <code>gpId</code>: 组号。范围为 0 至 n-1 ● <code>cnt</code>: 轴数。范围为 0 至 3
返回值	null

4.9.41 获取附加轴数

命令格式	<code>mot.getAuxAxisCount(int8_t gpId)</code>
命令功能	获取附加轴数
接口参数	● <code>gpId</code>: 组号。范围为 0 至 n-1
返回值	轴数

4.9.42 获取附加轴起始轴号

命令格式	<code>mot.getAuxAxisBegin(int8_t gpId)</code>
命令功能	获取附加轴起始轴号
接口参数	● <code>gpId</code>: 组号。范围为 0 至 n-1
返回值	起始轴号

4.9.43 配置协同组

命令格式	<code>mot.setRobCoordConfig(int8_t gpId, int8_t coordId, string axisId, string type)</code>
命令功能	配置协同组

接口参数	● gpId: 组号。范围为 0 至 n-1 ● coordId: 协同组号。范围为 0 至 n-1 ● axisId: 占用轴号。格式: 大括号扩起, 半角逗号分隔的数列, 例如: “{6,7,8,}”, 注意: 必须为 3 个数据 ● type: 占用轴用途。格式: 大括号扩起, 半角逗号分隔的数列, 例如 “{1,1,1,}”, 注意: 必须为 3 个数据
返回值	❖ 0: 表示配置未成功 ❖ 1: 表示配置成功

4.9.44 获取协同组配置

命令格式	mot.getRobCoordConfig(int8_t gpId, int8_t coordId)
命令功能	获取协同组配置
接口参数	● gpId: 组号。范围为 0 至 n-1 ● coordId: 协同组号。范围为 0 至 n-1
返回值	协同组配置 例如: “axisId={6,7,8,}, type={1,1,1,}” 其中, “axisId”是占用轴号, “type”是占用轴用途

4.9.45 获取轴组当前可协同运动轴的编号掩码

命令格式	mot.getCoordAxisMask(int8_t gpId)
命令功能	获取轴组当前可协同运动轴的编号掩码
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	可协同运动轴的编号掩码。低 9 位有效, 最低位代表组中第 1 个轴, 第 9 位代表组中第 9 个轴。值为 1 的轴为可协同运动

4.9.46 获取关节坐标点数据

命令格式	<code>mot.getJntData(int8_t gpId)</code>
命令功能	获取关节坐标点数据
接口参数	● <code>gpId</code> : 组号。范围为 0 至 <code>n-1</code>
返回值	点数据。格式：大括号扩起，半角逗号分隔的数列 例如：“{1.0,2.0,3.0,4.0,5.0,6.0,}”

4.9.47 获取附加轴坐标点数据

命令格式	<code>mot.getAuxData(int8_t gpId)</code>
命令功能	获取附加轴坐标点数据
接口参数	● <code>gpId</code> : 组号。范围为 0 至 <code>n-1</code>
返回值	点数据。格式：大括号扩起，半角逗号分隔的数列 例如：“{1.0,2.0,3.0,4.0,5.0,6.0,}”

4.9.48 获取关节反馈电流数据

命令格式	<code>mot.getJntEData(int8_t gpId)</code>
命令功能	获取关节反馈电流数据
接口参数	● <code>gpId</code> : 组号。范围为 0 至 <code>n-1</code>
返回值	点数据。格式：大括号扩起，半角逗号分隔的数列 例如：“{1.0,2.0,3.0,4.0,5.0,6.0,}”

4.9.49 获取附加轴反馈电流数据

命令格式	<code>mot.getAuxEData(int8_t gpId)</code>
命令功能	获取附加轴反馈电流数据
接口参数	● <code>gpId</code> : 组号。范围为 0 至 <code>n-1</code>

返回值	点数据。格式：大括号扩起，半角逗号分隔的数列 例如：“{1.0,2.0,3.0,4.0,5.0,6.0,}”
-----	---

4.9.50 获取笛卡尔坐标点数据

命令格式	mot.getLocData(int8_t gpId)
命令功能	获取笛卡尔坐标点数据
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	点数据。格式：大括号扩起，半角逗号分隔的数列 例如：“{1.0,2.0,3.0,4.0,5.0,6.0,}”

4.9.51 获取形态位

命令格式	mot.getConfig(int8_t gpId)
命令功能	获取形态位
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	形态位

4.9.52 检查点位是否可达

命令格式	mot.isReachable(int8_t gpId, bool isJoint, int8_t ufNum, int8_t utNum, int32_t config, string strPos, int8_t type)
命令功能	检查点位是否可达
接口参数	● gpId: 组号。范围为 0 至 n-1 ● isJoint: 是否关节点 ● ufNum: 工件号。范围为-1 至 15 ● utNum: 工具号。范围为-1 至 15 ● config: 形态位

	● strPos: 点数据。格式: 大括号扩起, 半角逗号分隔的数列, 例如: “{1.0,2.0,3.0,4.0,5.0,6.0,}” ● type: 类型
返回值	null

4.9.53 运动定位

命令格式	mot.moveTo(int8_t gpId, bool isJoint, int8_t ufNum, int8_t utNum, int32_t config, string strPos, bool isLinear)
命令功能	运动定位
接口参数	● gpId: 组号。范围为 0 至 n-1 ● isJoint: 是否关节点 ● ufNum: 工件号。范围为-1 至 15 ● utNum: 工具号。范围为-1 至 15 ● config: 形态位 ● strPos: 点数据。格式: 大括号扩起, 半角逗号分隔的数列, 例如: “{1.0,2.0,3.0,4.0,5.0,6.0,}” ● isLinear: 是否直线运动
返回值	null

4.9.54 圆弧运动定位

命令格式	mot.moveRad(int8_t gpId, string strMidPos, string strEndPos, bool isToMid)
命令功能	圆弧运动定位
接口参数	● gpId: 组号。范围为 0 至 n-1 ● strMidPos: 中间点位。格式: 关节点为 “{{1.0,2.0,3.0,4.0,5.0,6.0,}}” ; 空间点为 “{1,2,0,{1.0,2.0,3.0,4.0,5.0,6.0,}}” ● strEndPos: 末端点位

	● isToMid: 是否到中间点。true 表示运动到中间点；false 表示到运动到末端点
返回值	null

4.9.55 设置工件坐标系

命令格式	mot.setWorkpiece(int8_t gpId, int8_t index, string data)
命令功能	设置工件坐标系
接口参数	● gpId: 组号。范围为 0 至 n-1 ● index: 工件号。范围为 0 至 15 ● data: 点数据。格式：大括号扩起，半角逗号分隔的数列，例如： “{1.0,2.0,3.0,4.0,5.0,6.0,}”
返回值	null

4.9.56 获取工件坐标系

命令格式	mot.getWorkpiece(int8_t gpId, int8_t index)
命令功能	获取工件坐标系
接口参数	● gpId: 组号。范围为 0 至 n-1 ● index: 工件号。范围为 0 至 15
返回值	点数据。格式：大括号扩起，半角逗号分隔的数列 例如：“{1.0,2.0,3.0,4.0,5.0,6.0,}”

4.9.57 设置工具坐标系

命令格式	mot.setTool(int8_t gpId, int8_t index, string data)
命令功能	设置工具坐标系
接口参数	● gpId: 组号。范围为 0 至 n-1 ● index: 工件号。范围为 0 至 15

	● data: 点数据。格式: 大括号扩起, 半角逗号分隔的数列, 例如: “{1.0,2.0,3.0,4.0,5.0,6.0,}”
返回值	null

4.9.58 获取工具坐标系

命令格式	mot.getTool(int8_t gpId, int8_t index)
命令功能	获取工具坐标系
接口参数	● gpId: 组号。范围为 0 至 n-1 ● index: 工件号。范围为 0 至 15
返回值	点数据。格式: 大括号扩起, 半角逗号分隔的数列 例如: “{1.0,2.0,3.0,4.0,5.0,6.0,}”

4.9.59 设置安装工具方式

命令格式	mot.setInstallToolType(int8_t gpId, int8_t type)
命令功能	设置安装工具方式
接口参数	● gpId: 组号。范围为 0 至 n-1 ● type: 方式类型。0 表示法兰工具; 1 表示外部工具
返回值	null

4.9.60 获取安装工具方式

命令格式	mot.getInstallToolType(int8_t gpId)
命令功能	获取安装工具方式
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	❖ 0: 表示法兰工具 ❖ 1: 表示外部工具

4.9.61 设置协同组号

命令格式	mot.setCoordNum(int8_t gpId, int8_t num)
命令功能	设置协同组号
接口参数	● gpId: 组号。范围为 0 至 n-1 ● num: 协同组号。范围为-1 至 2
返回值	null

4.9.62 获取协同组号

命令格式	mot.getCoordNum(int8_t gpId)
命令功能	获取协同组号
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	协同组号

4.9.63 设置工件号

命令格式	mot.setWorkpieceNum(int8_t gpId, int8_t num)
命令功能	设置工件号
接口参数	● gpId: 组号。范围为 0 至 n-1 ● num: 工件号。范围为-1 至 15
返回值	null

4.9.64 获取工件号

命令格式	mot.getWorkpieceNum(int8_t gpId)
命令功能	获取工件号
接口参数	● gpId: 组号。范围为 0 至 n-1

返回值	工件号
-----	-----

4.9.65 设置工具号

命令格式	mot.setToolNum(int8_t gpId, int8_t num)
命令功能	设置工具号
接口参数	● gpId: 组号。范围为 0 至 n-1 ● num: 工具号。范围为-1 至 15
返回值	null

4.9.66 获取工具号

命令格式	mot.getToolNum(int8_t gpId)
命令功能	获取工具号
接口参数	● gpId: 组号。范围为 0 至 n-1
返回值	工具号

4.9.67 标定

命令格式	mot.gpCalibrate(string what, int8_t gpId, int8_t num, int8_t type, string strData)
命令功能	标定。标定成功后数据即可生效，但不会保存
接口参数	● what: 标定对象。"BASE"表示工件；"TOOL"表示工具；"EXT_TOOL"表示外部工具；"EXT_BASE"表示外部工件；"COORD"表示协同组 ● gpId: 组号。范围为 0 至 n-1 ● num: 工件号/工具号。范围为-1 至 15 ● type: 标定方法编号。6 表示工具 6 点法；5 表示协同组 5 点法；4 表示工具 4 点法；3 表示工件 3 点法/协同组 3 点法/外部工具 3 点法/外部工件 3 点法；2 表示工具 2 点法

	● strData: 标定数据。格式: 半角分号分隔的点数据队列, 例如: “{1,2},{3,4},{5,6}”
返回值	标定结果 例如: “ret=0x0, data={7,8,}” 其中, “ret”是标定返回值, “data”是标定结果

4.9.68 标定附加轴减速比

命令格式	mot.calibrateAuxAxisGearRatio(string strJsonInput)
命令功能	标定附加轴减速比。标定成功后数据即可生效, 但不会保存
接口参数	● strJsonInput: JSON 格式标定输入内容。格式: {"gpId":0, "auxAxId":6, "auxAxType":"L", "sampleList":[{"xyz":[123.0, ...], "enc":123.0}, ...]}, 其中"gpId"表示组号, 范围为 0 至 n-1; "auxAxId"表示附加轴号; 范围为 0 至 2; "auxAxType"表示附加轴类型, "L"表示线性, "R"表示旋转; "sampleList"表示标定数据样本列表, 旋转轴需 3 个样本; "xyz"表示样本 XYZ, 共 3 个坐标值; "enc"表示样本附加轴编码器值
返回值	减速比

4.9.69 获取轴减速比

命令格式	mot.getGearRatio(int8_t gpId, int8_t axId)
命令功能	获取轴减速比
接口参数	● gpId: 组号。范围为 0 至 n-1 ● axId: 组内轴号。范围为 0 至 8
返回值	减速比

4.9.70 20 点标定

命令格式	mot.calibrateWith20Points(string strJsonInput)
------	--

命令功能	20 点标定。标定成功后数据即可生效，但不会保存
接口参数	● strJsonInput: JSON 格式标定输入内容。格式: {"gpId":0, "utNum":0, "sampleList":[{"jnt":[123.0, ...]}, ...]}, 其中"gpId"表示组号, 范围为 0 至 n-1; "utNum"表示工具号, 范围为 0 至 15; "sampleList"表示标定数据样本列表, 需 10 至 20 个样本; "jnt"表示样本关节坐标, 1 至 9 个值
返回值	JSON 格式标定结果 格式: {"ec":0, "UT":"****", "zero":"****"} 其中, "ec"表示 64 位错误码; "UT"表示工具值字符串; "zero"表示关节零点值字符串

4.10 业务功能代理 lib

4.10.1 获取业务层版本信息

命令格式	lib.versions()
命令功能	获取业务层版本信息
接口参数	null
返回值	版本信息字符串

4.10.2 设置控制器名字

命令格式	lib.setName(string name)
命令功能	设置控制器名字
接口参数	● name: 名字
返回值	null

4.10.3 获取控制器版本信息

命令格式	lib.getControllerVer()
------	------------------------

命令功能	获取控制器版本信息
接口参数	null
返回值	控制器总体版本信息

4.10.4 关机

命令格式	lib.shutdown()
命令功能	关闭控制系统
接口参数	null
返回值	null

4.10.5 重启

命令格式	lib.reboot()
命令功能	重启控制系统
接口参数	null
返回值	null

4.10.6 获取控制器信息

命令格式	lib.getIpcInfo()
命令功能	获取控制器信息
接口参数	null
返回值	控制器信息

4.10.7 将内存数据同步至硬盘

命令格式	lib.syncDisk()
------	----------------

命令功能	将内存数据同步至硬盘
接口参数	null
返回值	null

4.10.8 清理硬盘数据

命令格式	lib.clearDisk()
命令功能	清理硬盘数据
接口参数	null
返回值	null

4.11 数字 IO 操作代理 io

4.11.1 获取数字输入端口数量

命令格式	io.getMaxDinNum()
命令功能	获取数字输入端口数量
接口参数	null
返回值	数量

4.11.2 获取数字输出端口数量

命令格式	io.getMaxDoutNum()
命令功能	获取数字输出端口数量
接口参数	null
返回值	数量

4.11.3 获取数字输入分组数量

命令格式	io.getMaxDinGrp()
命令功能	获取数字输入分组数量。32 个输入为一组，组 0 包括数字输入 0 至 31，组 1 包括数字输入 32 至 63，往后以此类推
接口参数	null
返回值	数量

4.11.4 获取数字输出分组数量

命令格式	io.getMaxDoutGrp()
命令功能	获取数字输出分组数量。32 个输出为一组，组 0 包括数字输出 0 至 31，组 1 包括数字输出 32 至 63，往后以此类推
接口参数	null
返回值	数量

4.11.5 获取数字输入分组值

命令格式	io.getDinGrp(int32_t grpIndex)
命令功能	获取数字输入分组值
接口参数	grpIndex: 分组号。范围为 0 至 n-1
返回值	分组值。32 位无符号整数，最低位代表分组中数字输入 0 的值，最高位代表数字输入 31 的值

4.11.6 获取数字输出分组值

命令格式	io.getDoutGrp(int32_t grpIndex)
命令功能	获取数字输出分组值
接口参数	grpIndex: 分组号。范围为 0 至 n-1

返回值	分组值。32 位无符号整数，最低位代表分组中数字输出 0 的值，最高位代表数字输出 31 的值
-----	---

4.11.7 获取数字输入分组状态

命令格式	io.getDinMaskGrp(int32_t grpIndex)
命令功能	获取数字输入分组状态
接口参数	grpIndex: 分组号。范围为 0 至 n-1
返回值	分组状态。32 位无符号整数，最低位代表分组中数字输入 0 的状态，最高位代表数字输入 31 的状态

4.11.8 获取数字输出分组状态

命令格式	io.getDoutMaskGrp(int32_t grpIndex)
命令功能	获取数字输出分组状态
接口参数	grpIndex: 分组号。范围为 0 至 n-1
返回值	分组状态。32 位无符号整数，最低位代表分组中数字输出 0 的状态，最高位代表数字输出 31 的状态

4.11.9 设置数字输入端口值

命令格式	io.setDin(int32_t portIndex, bool value)
命令功能	设置数字输入端口值
接口参数	- portIndex: 端口号。范围为 0 至 n-1 - value: 值
返回值	null

4.11.10 设置数字输出端口值

命令格式	io.setDout(int32_t portIndex, bool value)
命令功能	设置数字输出端口值
接口参数	● portIndex: 端口号。范围为 0 至 n-1 ● value: 值
返回值	null

4.11.11 获取数字输入端口值

命令格式	io.getDin(int32_t portIndex)
命令功能	获取数字输入端口值
接口参数	● portIndex: 端口号。范围为 0 至 n-1
返回值	值

4.11.12 获取数字输出端口值

命令格式	io.getDout(int32_t portIndex)
命令功能	获取数字输出端口值
接口参数	● portIndex: 端口号。范围为 0 至 n-1
返回值	值

4.11.13 设置数字输入状态值

命令格式	io.setDinMaskBit(int32_t portIndex, bool stat)
命令功能	设置数字输入状态值。数字输入状态为“VIRTUAL”时，可通过 setDin 设置值；数字输入状态为“REAL”时，值从 IO 模块中读取，通过 setDin 设置值无效
接口参数	● portIndex: 端口号。范围为 0 至 n-1 ● stat: 状态。true 表示虚拟；false 表示真实

返回值	null
-----	------

4.11.14 设置数字输出状态值

命令格式	io.setDoutMaskBit(int32_t portIndex, bool stat)
命令功能	设置数字输出状态值。数字输出状态为“VIRTUAL”时，可通过 setDout 设置值，但不能触发 IO 模块输出；数字输出状态为“REAL”时，可通过 setDout 设置值，并可触发 IO 模块输出
接口参数	- portIndex: 端口号。范围为 0 至 n-1 - stat: 状态。true 表示虚拟；false 表示真实
返回值	null

4.12 零点操作代理 home

4.12.1 设置零点

命令格式	home.setHome(int gpID, string home, int mask)
命令功能	设置零点
接口参数	- gpID: 组号。范围为 0 至 n-1 - home: 零点。例如: "{0,-90,180,0,90,0,0,0}" - mask: 掩码
返回值	null

4.12.2 获取零点

命令格式	home.getHome(int gpID)
命令功能	获取零点
接口参数	gpID: 组号。范围为 0 至 n-1

返回值	点数据。格式：大括号扩起，半角逗号分隔的数列 例如： "{1.0,2.0,3.0,4.0,5.0,6.0,0.0,0.0,0.0}"
-----	---

4.12.3 通过零点偏差计算零点

命令格式	home.setHomeByOffset(int gpID, string home, string offset, int mask)
命令功能	通过零点偏差计算零点。在编码器值丢失时可使用
接口参数	● gpID: 组号。范围为 0 至 n-1 ● home: 零点。例如： "{0,-90,180,0,90,0,0,0,0}" ● offset: 零点偏移。例如： "{0,0,0,0,0,0,0,0,0}" ● mask: 掩码
返回值	null

4.12.4 获取零点偏移

命令格式	home.getHomeOffset(int gpID)
命令功能	获取零点偏移
接口参数	● gpID: 组号。范围为 0 至 n-1
返回值	点数据。格式：大括号扩起，半角逗号分隔的数列 例如： "{1.0,2.0,3.0,4.0,5.0,6.0,0.0,0.0,0.0}"

4.12.5 通过编码器值计算零点

命令格式	home.setHomeByEncoder(int gpID, string home, int mask)
命令功能	通过编码器值计算零点
接口参数	● gpID: 组号。范围为 0 至 n-1 ● home: 零点。例如： "{0,-90,180,0,90,0,0,0,0}" ● mask: 掩码

返回值	null
-----	------

4.13 外部运行功能代理 ext

4.13.1 添加外部输入信号

命令格式	ext.addExtIn(int iDin, string sig)
命令功能	添加外部输入信号
接口参数	● iDin: 外部输入信号索引。范围为 0 至 n-1 ● sig: 外部输入信号名
返回值	null

4.13.2 删除外部输入信号

命令格式	ext.delExtIn(int iDin, string sig)
命令功能	删除外部输入信号
接口参数	● iDin: 外部输入信号索引。范围为 0 至 n-1 ● sig: 外部输入信号名
返回值	null

4.13.3 获取外部输入信号配置列表

命令格式	ext.getExtInList()
命令功能	获取外部输入信号配置列表
接口参数	null
返回值	外部输入信号配置列表。格式：中括号扩起，半角逗号分隔的数据队列 例如： “["iDin=1, sig="iENABLE"", ...]” 其中，“iDin”是外部输入信号索引，“sig”是外部输入信号名

4.13.4 添加外部输出信号

命令格式	ext.addExtOut(int iDout, string sig)
命令功能	添加外部输出信号
接口参数	● iDout: 外部输出信号索引。范围为 0 至 n-1 ● sig: 外部输出信号名
返回值	null

4.13.5 删除外部输出信号

命令格式	ext.delExtOut(int iDout, string sig)
命令功能	删除外部输出信号
接口参数	● iDout: 外部输出信号索引。范围为 0 至 n-1 ● sig: 外部输出信号名
返回值	null

4.13.6 获取外部输出信号配置列表

命令格式	ext.getExtOutList()
命令功能	获取外部输出信号配置列表
接口参数	null
返回值	外部输出信号配置列表。格式：中括号扩起，半角逗号分隔的配置数据队列 例如：“["iDout=1, sig="oENABLE_STATE", ...]” 其中，“iDout”是外部输出信号索引，“sig”是外部输出信号名

4.13.7 修改引用寄存器配置

命令格式	ext.modRefRegData(int iRef, int iGroup, string typeReg, int iReg, double precision)
命令功能	修改引用寄存器配置

接口参数	● iRef: 引用信号索引 ● iGroup: 绑定组号 ● typeReg: 绑定寄存器类型。"JR"表示 JR 寄存器 ● iReg: 绑定寄存器索引 ● precision: 精度
返回值	null

4.13.8 删除引用寄存器配置

命令格式	ext.delRefRegData(int iRef)
命令功能	删除引用寄存器配置
接口参数	● iRef: 引用信号索引
返回值	null

4.13.9 获取引用寄存器配置

命令格式	ext.getRefRegData(int32_t iRef)
命令功能	获取引用寄存器配置
接口参数	● iRef: 引用信号索引
返回值	引用寄存器配置 例如: “iRef=1, iGroup=0, typeReg="JR", iReg=99, precision=1.000000” 其中, “iRef” 是引用信号索引, “sig” 是外部输出信号名, “iGroup” 是绑定组号, “typeReg” 是绑定寄存器类型, “iReg” 是绑定寄存器索引, “precision” 是精度

4.13.10 获取引用寄存器配置列表

命令格式	ext.getRefRegDataInfo()
命令功能	获取引用寄存器配置列表

接口参数	null
返回值	引用寄存器配置列表。格式：中括号扩起，半角逗号分隔的数据队列 例如：“["iRef=1, iGroup=0, typeReg="JR", iReg=99, precision=1.000000", ...]” 其中，“iRef”是引用信号索引，“sig”是外部输出信号名，“iGroup”是绑定组号，“typeReg”是绑定寄存器类型，“iReg”是绑定寄存器索引，“precision”是精度

4.13.11 设置外部运行程序

命令格式	ext.setExtProgName(int index, string entry)
命令功能	设置外部运行程序
接口参数	● index：配置项编号 ● entry：入口程序路径
返回值	null

4.13.12 获取外部运行程序映射表

命令格式	ext.getExtProgNameMap()
命令功能	获取外部运行程序映射表
接口参数	null
返回值	映射表。格式：中括号扩起，半角逗号分隔的外部运行程序配置映射表 例如：“[1=X.PRG, 2=/a/Y.PRG]”

4.13.13 设置外部运行程序编号绑定寄存器

命令格式	ext.setR4ExtProg(int iR)
命令功能	设置外部运行程序编号绑定寄存器
接口参数	● iR：R 寄存器索引。范围为 0 至 n-1
返回值	null

4.13.14 获取外部运行程序编号绑定寄存器

命令格式	ext.getR4ExtProg()
命令功能	获取外部运行程序编号绑定寄存器
接口参数	null
返回值	R 寄存器索引

4.13.15 保存钩子相关内容

命令格式	ext.saveHook()
命令功能	保存涉及编码/解码配置、外部输入/输出配置的内容
接口参数	null
返回值	null

4.13.16 保存

命令格式	ext.save()
命令功能	保存涉及引用寄存器配置、外部运行程序的内容
接口参数	null
返回值	null

4.14 动力学功能代理 dyn

4.14.1 设置协作模式

命令格式	dyn.setDynamicMode(bool en)
命令功能	设置协作模式
接口参数	null

返回值	null
-----	------

4.14.2 获取协作模式

命令格式	dyn.getDynamicMode()
命令功能	获取协作模式
接口参数	null
返回值	null

4.14.3 初始化动力学辨识模块

命令格式	dyn.initIdentification(int8_t sampleNum, int8_t isLoadIden)
命令功能	初始化动力学辨识模块
接口参数	● sampleNum: 激励轨迹运行次数 ● isLoadIden: 是否为负载辨识
返回值	null

4.14.4 设置负载辨识关节运动范围

命令格式	dyn.genLoadIdenTraj(string strLimit)
命令功能	设置负载辨识关节运动范围
接口参数	● strLimit: 轴运动范围。格式：半角分号分隔的点数据队列，例如： “{2,10,-10};{3,10,-10};{4,10,-10};{5,10,-10}”，其中{2,10,-10}为三轴正负限位， 往后依次为四轴、五轴、六轴正负限位
返回值	null

4.14.5 获取负载激励轨迹系数

命令格式	dyn.getLoadIdenTrajCoff()
------	---------------------------

命令功能	获取负载激励轨迹系数
接口参数	null
返回值	轨迹系数。格式：大括号扩起，半角逗号分隔的数列 例如： "{1.0,2.0,3.0,4.0,5.0,6.0,}"

4.14.6 运行激励轨迹

命令格式	dyn.runIdenTraj(int8_t trajCnt, int8_t sampleCnt)
命令功能	运行激励轨迹
接口参数	● trajCnt: 激励轨迹序号 ● sampleCnt: 激励轨迹运行序号
返回值	null

4.14.7 低速运行激励轨迹

命令格式	dyn.runIdenTrajSafely(int8_t trajCnt)
命令功能	低速运行激励轨迹
接口参数	● trajCnt: 激励轨迹序号
返回值	null

4.14.8 停止激励轨迹运行

命令格式	dyn.stopCurrentTraj()
命令功能	停止激励轨迹运行
接口参数	null
返回值	null

4.14.9 获取辨识轨迹起始点

命令格式	dyn.getStartPos(int8_t trajCnt)
命令功能	获取辨识轨迹起始点
接口参数	● trajCnt: 激励轨迹序号
返回值	关节位置。格式：大括号扩起，半角逗号分隔的数列 例如: "{1.0,2.0,3.0,4.0,5.0,6.0,}"

4.14.10 执行动力学辨识

命令格式	dyn.identify()
命令功能	执行动力学辨识
接口参数	null
返回值	null

4.14.11 执行负载辨识

命令格式	dyn.identifyLoad(int8_t num)
命令功能	执行负载辨识
接口参数	● num: 负载编号
返回值	null

4.14.12 获取激励轨迹运行状态

命令格式	dyn.getIdenTrajState()
命令功能	获取激励轨迹运行状态
接口参数	null
返回值	❖ 0: 表示激励轨迹运行完成 ❖ 1: 表示激励轨迹运行中

	❖ 2: 表示激励轨迹运行失败
--	-----------------

4.14.13 负载切换

命令格式	dyn.switchLoad(int8_t loadNum)
命令功能	负载切换
接口参数	loadNum: 负载序号
返回值	null

4.14.14 清除负载

命令格式	dyn.clearLoad()
命令功能	清除负载。即切换为空载
接口参数	null
返回值	null

4.14.15 切换动力学模块

命令格式	dyn.switchMode(int8_t mode)
命令功能	切换动力学模块
接口参数	mode: 动力学模块。0 表示拖拽模式；1 表示位置控制模式（力矩前馈模式），此为默认模式；2 表示碰撞响应模式
返回值	null

4.14.16 设置碰撞灵敏度

命令格式	dyn.setCollisionSen(int8_t axis, int8_t sens)
命令功能	设置碰撞灵敏度

接口参数	● axis: 轴号 ● sens: 碰撞灵敏度
返回值	null

4.14.17 获取碰撞灵敏度

命令格式	dyn.getCollisionSen(int8_t axis)
命令功能	获取碰撞灵敏度
接口参数	● axis: 轴号
返回值	碰撞灵敏度

4.15 采集操作代理 collect

4.15.1 配置采集任务

命令格式	collect.config(int32_t iSessionId, string strConfig)
命令功能	配置采集任务
接口参数	● iSessionId: 网络连接 ID ● strConfig: 配置信息
返回值	null

4.15.2 添加采集任务

命令格式	collect.addTask(int32_t iSessionId, int32_t tid, int32_t iSampleKT, string strVarList)
命令功能	添加采集任务
接口参数	● iSessionId: 网络连接 ID ● tid: 任务 ID ● iSampleKT: 采集周期的倍数

	● strVarList: 变量字符串列表
返回值	null

4.15.3 删除采集任务

命令格式	collect.delTask(int32_t iSessionId, int32_t tid)
命令功能	删除采集任务
接口参数	● iSessionId: 网络连接 ID ● tid: 任务 ID
返回值	null

4.15.4 启动所有采集任务

命令格式	collect.start(int32_t iSessionId)
命令功能	启动所有采集任务
接口参数	● iSessionId: 网络连接 ID
返回值	null

4.15.5 启动采集任务

命令格式	collect.start(int32_t iSessionId, int32_t tid)
命令功能	启动采集任务
接口参数	● iSessionId: 网络连接 ID ● tid: 任务 ID
返回值	null

4.15.6 停止所有采集任务

命令格式	collect.stop(int32_t iSessionId)
------	----------------------------------

命令功能	停止所有采集任务
接口参数	● iSessionId: 网络连接 ID
返回值	null

4.15.7 停止采集任务

命令格式	collect.stop(int32_t iSessionId, int32_t tid)
命令功能	停止采集任务
接口参数	● iSessionId: 网络连接 ID ● tid: 任务 ID
返回值	null

4.15.8 获取存在的采集任务列表

命令格式	collect.getExistTasks(int32_t iSessionId)
命令功能	获取存在的采集任务列表
接口参数	● iSessionId: 网络连接 ID
返回值	任务号 tid 列表。格式：中括号扩起，半角逗号分隔的队列 例如：“[1, 2, 3]”

4.15.9 获取正在运行的采集任务列表

命令格式	collect.getRunningTasks(int32_t iSessionId)
命令功能	获取正在运行的采集任务列表
接口参数	● iSessionId: 网络连接 ID
返回值	任务号 tid 列表。格式：中括号扩起，半角逗号分隔的队列 例如：“[1, 2, 3]”

4.15.10 获取当前时间戳

命令格式	collect.getCurTimestamp()
命令功能	获取当前时间戳
接口参数	null
返回值	时间戳（滴答数）

4.16 编码解码功能代理 coder

4.16.1 添加编码器

命令格式	coder.addEncoder(int32_t iDout, int32_t lenDout, int32_t iR)
命令功能	添加编码器
接口参数	● iDout: 数字输出起始索引。范围为 0 至 n-1 ● lenDout: 数字输出位数。范围为 1 至 n ● iR: R 寄存器索引。范围为 0 至 n-1
返回值	null

4.16.2 删除编码器

命令格式	coder.delEncoder(int32_t iDout, int32_t lenDout, int32_t iR)
命令功能	删除编码器
接口参数	● iDout: 数字输出起始索引。范围为 0 至 n-1 ● lenDout: 数字输出位数。范围为 1 至 n ● iR: R 寄存器索引。范围为 0 至 n-1
返回值	null

4.16.3 获取编码器列表

命令格式	<code>coder.getEncoderList()</code>
命令功能	获取编码器列表
接口参数	null
返回值	解码器列表。格式：中括号扩起，半角逗号分隔的解码器数据队列 例如：“["iDout=31, lenDout=3, iR=5", ...]” 其中，“iDout”是 DOUT 起始索引，“lenDout”是 DOUT 位数，“iR”是 R 寄存器索引

4.16.4 添加解码器

命令格式	<code>coder.addDecoder(int iR, int iDin, int lenDin)</code>
命令功能	添加解码器
接口参数	● iR: R 寄存器索引。范围为 0 至 n-1 ● iDin: 数字输入起始索引。范围为 0 至 n-1 ● lenDin: 数字输入位数。范围为 1 至 n
返回值	null

4.16.5 删除解码器

命令格式	<code>coder.delDecoder(int iR, int iDin, int lenDin)</code>
命令功能	删除解码器
接口参数	● iR: R 寄存器索引。范围为 0 至 n-1 ● iDin: 数字输入起始索引。范围为 0 至 n-1 ● lenDin: 数字输入位数。范围为 1 至 n
返回值	null

4.16.6 获取解码器列表

命令格式	<code>coder.getDecoderList()</code>
命令功能	获取解码器列表
接口参数	null
返回值	解码器列表。格式：中括号扩起，半角逗号分隔的解码器数据队列 例如：“["iR=1, iDin=2, lenDin=3", ...]” 其中，“iR”是R寄存器索引，“iDin”是DIN起始索引，“lenDin”是DIN位数

4.16.7 保存

命令格式	<code>coder.save()</code>
命令功能	保存涉及编码/解码配置、外部输入/输出配置的内容
接口参数	null
返回值	null

4.17 标定操作代理 cali

4.17.1 协同组标定

命令格式	<code>cali.calibrate(int8_t gpId, int8_t num, int8_t type, string strData)</code>
命令功能	协同组标定
接口参数	● gpId: 组号。范围为0至n-1 ● num: 协同组号。范围为-1至2 ● type: 标定方法编号。5表示5点法；3表示3点法 ● strData: 标定数据。格式：半角分号分隔的点数据队列，例如： “{1,2},{3,4},{5,6}”
返回值	标定结果 例如：“ret=0x0, data={{7,8},{9,10}}”

	其中，“ret”是标定返回值，“data”是标定结果，是一个二维矩阵
--	------------------------------------

4.17.2 外部轴减速比标定

命令格式	cali.caliExtAxisRatio(bool isLine, string strCali, string strEnc)
命令功能	外部轴减速比标定
接口参数	● isLine: 是否直线轴。true 表示直线轴；false 表示旋转轴 ● strCali: 标定数据。格式：半角分号分隔的点数据队列，例如：“{1,2},{3,4},{5,6}” ● strEnc: 编码器数据。格式：半角分号分隔的点数据队列，例如：“{1,2,3}”
返回值	标定结果

4.17.3 外部工具标定

命令格式	cali.caliExtTool(string strCali)
命令功能	外部工具标定
接口参数	● strCali: 标定数据。格式：半角分号分隔的点数据队列，例如：“{1,2},{3,4},{5,6}”
返回值	标定结果 例如：“ret=0x0, data={7,8,}” 其中，“ret”是标定返回值，“data”是标定结果

4.17.4 外部工件标定

命令格式	cali.caliExtWorkPiece(string strCali)
命令功能	外部工件标定
接口参数	● strCali: 标定数据。格式：半角分号分隔的点数据队列，例如： “{1,2},{3,4},{5,6},{7,8,}”，其中{1,2,}表示空间点 1；{3,4,}表示空间点 2；{5,6,} 表示空间点 3；{7,8,}表示外部工具标定值（XYZABC）
返回值	标定结果

	例如：“ret=0x0, data={9,10,}” 其中，“ret”是标定返回值，“data”是标定结果
--	---

4.18 算法功能代理 Algo

4.18.1 关节坐标转换空间坐标 1

命令格式	Algo.calcJPosToCPos(int ToolNum, int WorkNum, string strJPos)
命令功能	关节坐标转换空间坐标
接口参数	● ToolNum: 工具号 ● WorkNum: 工件号 ● strJPos: 关节角。格式：半角分号分隔的点数据队列，例如：“{1,2,}”
返回值	标定结果 例如：“ret=0x0, data={{3,4},{5,6,}}” 其中，“ret”是标定返回值，“data”是标定结果，{3,4,}是法兰盘在基坐标位置，{5,6,}是工具点在工件中位置

4.18.2 关节坐标转换空间坐标 2

命令格式	Algo.calcCPosToJPosByApproach(int ToolNum, int WorkNum, int Cfg, string strCPos, string strJPos)
命令功能	关节坐标转换空间坐标。该命令为临近关节角求解
接口参数	● ToolNum: 工具号 ● WorkNum: 工件号 ● Cfg: 形态位 ● strCPos: 空间角。格式：半角分号分隔的点数据队列，例如：“{1,2,}” ● strJPos: 关节角。格式：半角分号分隔的点数据队列，例如：“{3,4,}”
返回值	标定结果

	例如：“ret=0x0, data={5,6,}” 其中，“ret”是标定返回值，“data”是标定结果
--	--

4.18.3 关节坐标转换空间坐标 3

命令格式	Algo.calcCPosToJPosByAnalysis(int ToolNum, int WorkNum, int State, int Cfg, string strCPos)
命令功能	关节坐标转换空间坐标。该命令为解析解求解
接口参数	● ToolNum: 工具号 ● WorkNum: 工件号 ● State: 空间位置的类型。0 表示法兰在基础坐标系中的空间位置；1 表示工具点在基础坐标系中的空间位置；2 表示工具点在工作坐标系中的空间位置 ● Cfg: 形态位 ● strCPos: 空间角。格式：半角分号分隔的点数据队列，例如：“{1,2,}”
返回值	标定结果 例如：“ret=0x0, data={3,4,}” 其中，“ret”是标定返回值，“data”是标定结果

4.19 模拟 IO 操作代理 ain/aout

4.19.1 获取 IO 变量数量

命令格式	ain/aout.getSize()
命令功能	获取变量框架支持的最大 IO 变量数量
接口参数	null
返回值	数量

4.19.2 获取真实端口数量

命令格式	ain/aout.getRealSize()
命令功能	获取真实端口数量
接口参数	null
返回值	数量

4.19.3 获取端口值

命令格式	ain/aout.getValue(size_t index)
命令功能	获取端口值
接口参数	● index: 端口号。范围为 0 至 n-1
返回值	值

4.19.4 设置端口值

命令格式	ain/aout.setValue(size_t index, float value)
命令功能	设置端口值
接口参数	● index: 端口号。范围为 0 至 n-1 ● value: 值
返回值	null

4.19.5 获取端口状态

命令格式	ain/aout.getMask(size_t index)
命令功能	获取端口状态。当状态为“VIRTUAL”时，通过 setValue 设置的值存放在进程内存中；当状态为“REAL”时，通过 setValue 设置的值存放在变量框架（共享内存）中
接口参数	● index: 端口号。范围为 0 至 n-1
返回值	❖ true: 表示虚拟状态

	❖ false: 表示真实状态
--	-----------------

4.19.6 设置端口状态

命令格式	ain/aout.setMask(size_t index, bool bVirtual)
命令功能	设置端口状态。当状态为“VIRTUAL”时，通过 setValue 设置的值存放在进程内存中；当状态为“REAL”时，通过 setValue 设置的值存放在变量框架（共享内存）中
接口参数	● index: 端口号。范围为 0 至 n-1 ● bVirtual: 状态。true 表示虚拟状态；false 表示真实状态
返回值	null

4.19.7 获取 IO 分组状态

命令格式	ain/aout.getMaskGrp(size_t grpIndex)
命令功能	获取 IO 分组状态。32 个端口为 1 组
接口参数	● grpIndex: 分组号。范围为 0 至 n-1
返回值	分组状态。32 位无符号整数，最低位代表分组中端口 0 的状态，最高位代表端口 31 的状态 ❖ true: 表示虚拟状态 ❖ false: 表示真实状态

4.19.8 设置 IO 分组状态

命令格式	ain/aout.setMaskGrp(size_t grpIndex, uint32_t mask)
命令功能	设置 IO 分组状态。32 个端口为 1 组
接口参数	● grpIndex: 分组号。范围为 0 至 n-1 ● mask: 分组状态。32 位无符号整数，最低位代表分组中端口 0 的状态，最高位代表端口 31 的状态。true 表示虚拟状态；false 表示真实状态

返回值	错误码 ❖ ERR_VAR_MGR_OK: 表示正常 ❖ 其他: 表示异常
-----	--